

Legal Rights Navigator

Document & Legal Knowledge Ingestion Plan

Covering all five supported legal domains — sourcing, structuring, chunking, embedding, and storage — with concrete Python implementation.

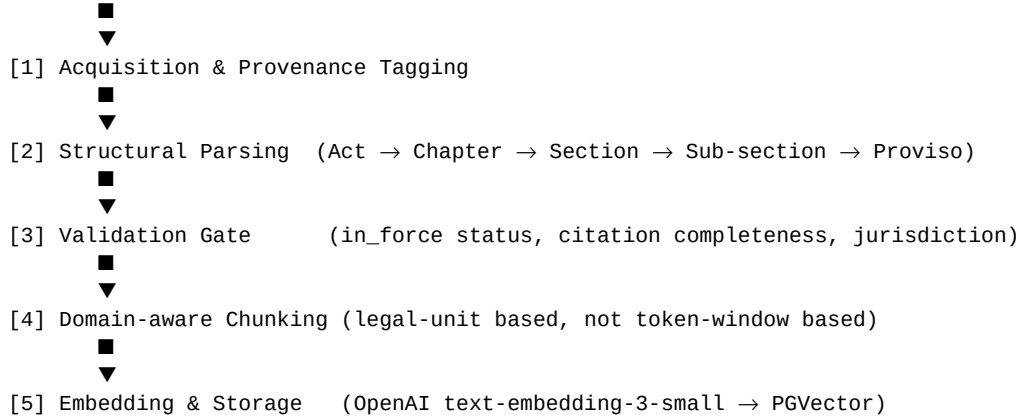
Scope: Labour & Employment · Consumer Protection · Tenant & Property · Cyber Crime · Family & Women's Rights

Stack: FastAPI · LangGraph · PostgreSQL + PGVector · OpenAI text-embedding-3-small

1. Ingestion Pipeline Overview

Every domain follows the same five-stage ingestion pipeline. Only the source list, metadata schema extensions, and a few parsing rules differ per domain. This keeps ingestion modular and testable, matching the project's core design principles.

Raw Source (Bare Act / Judgment / Circular / Notification)



1.1 Shared Metadata Schema (all domains)

Every chunk, regardless of domain, carries this base metadata. Domain-specific fields are added on top (see each domain section below).

```
class BaseChunkMetadata(BaseModel):
    chunk_id: str
    domain: Literal[
        "labour_employment", "consumer_protection", "tenant_property",
        "cyber_crime", "family_womens_rights"
    ]
    doc_type: Literal["statute", "judgment", "notification", "circular", "rule"]
    act_name: str
    act_year: int | None
    section_number: str | None
    section_title: str | None
    jurisdiction: Literal["central", "state"]
    state: str | None # null if central
    in_force: bool
    superseded_by: str | None # e.g. a Labour Code provision not yet notified
    source_url: str
    last_verified: date
    applicability_tags: list[str] # domain-specific, see below
```

2. Labour & Employment

Sources to ingest

- Central Acts: Industrial Disputes Act 1947, Payment of Wages Act 1936, Minimum Wages Act 1948, Payment of Gratuity Act 1972, EPF Act 1952, ESI Act 1948, Maternity Benefit Act 1961, Contract Labour (Regulation & Abolition) Act 1970
- New Labour Codes 2019/2020 (Wage Code, IR Code, Social Security Code, OSH Code) — tag every section with notified vs. not-yet-notified status
- State Shops & Establishments Acts (varies by state)
- Landmark SC/HC judgments on retrenchment, termination, wage disputes
- Ministry of Labour circulars, e-Gazette notifications on Code enforcement

Domain-specific metadata: applicability_tags

Added to the shared schema's `applicability_tags` field, used for sub-filtering after domain filtering and before semantic search.

- `organized_sector` / `unorganized_sector`
- `employment_type`: permanent, contract, gig, apprentice
- `establishment_size_threshold` (many provisions apply only above N employees)

Chunking rules specific to this domain

- One chunk per section for short sections (<500 tokens); one chunk per sub-section for long ones (e.g. gratuity computation rules) — always prepend parent Act + Section number as context.
- Definitions in Section 2 of each Act (e.g. 'workman', 'wages', 'retrenchment') are chunked individually — these are retrieved on their own very frequently ('am I a workman?').
- Provisos and explanations stay attached to their parent clause in the same chunk.
- Every chunk from the new Labour Codes must carry `in_force` + `superseded_by` status explicitly — this is the highest hallucination risk in this domain.

Python: domain-specific ingestion notes

```
def parse_labour_act_section(raw_section: dict, act_meta: dict) -> list[Chunk]:
    """Split one parsed Act section into chunk-ready units."""
    chunks = []
    header = f"{act_meta['act_name']} ({act_meta['act_year']}), " \
            f"Section {raw_section['number']} - {raw_section['title']}"

    if token_count(raw_section["text"]) < 500:
        chunks.append(make_chunk(header, raw_section["text"], raw_section, act_meta))
    else:
        for sub in raw_section["sub_sections"]:
            text = sub["text"] + "\n" + "\n".join(sub.get("provisos", []))
            chunks.append(make_chunk(f"{header} ({sub['label']})", text, raw_section, act_meta))
    return chunks
```

3. Consumer Protection

Sources to ingest

- Consumer Protection Act 2019 (and 2020 rules: e-commerce, product liability, unfair trade practices)

- Consumer Protection (E-Commerce) Rules 2020
- District/State/National Consumer Disputes Redressal Commission landmark orders
- Bureau of Indian Standards (BIS) product-safety notifications where relevant to claims
- Legal Metrology Act 2009 (for mislabeling / MRP-related complaints)

Domain-specific metadata: applicability_tags

Added to the shared schema's `applicability_tags` field, used for sub-filtering after domain filtering and before semantic search.

- `transaction_type`: goods, services, e-commerce, real_estate
- `complaint_value_bracket` (jurisdiction of District vs State vs National Commission depends on this)
- `grievance_type`: defective_good, deficiency_in_service, unfair_trade_practice, misleading_ad

Chunking rules specific to this domain

- Chunk the pecuniary jurisdiction table (District/State/National Commission value thresholds) as ONE dedicated chunk — it's a frequently-needed lookup and must not be split across chunks.
- E-commerce rules: chunk by obligation-holder (marketplace entity vs inventory e-commerce entity) since users' rights differ by which type of platform they dealt with.
- Commission orders: chunk by issue/ratio, always retain forum level (District/State/National) in metadata since precedent weight differs.

Python: domain-specific ingestion notes

```
JURISDICTION_TABLE_CHUNK_ID = "cp_pecuniary_jurisdiction_table_v1"
# This single, hand-verified chunk is force-included whenever the classifier
# detects a "which forum do I file in" style query, in addition to normal retrieval.
```

```
def tag_ecommerce_obligation(section: dict) -> str:
    text = section["text"].lower()
    if "marketplace" in text:
        return "marketplace_entity"
    if "inventory" in text:
        return "inventory_entity"
    return "general"
```

4. Tenant & Property

Sources to ingest

- State Rent Control Acts (materially different state to state — e.g. Delhi Rent Control Act 1958, Maharashtra Rent Control Act 1999)
- Model Tenancy Act 2021 (advisory/model law — states adopt selectively, must be flagged as such)
- Transfer of Property Act 1882 (central, general property transfer provisions)
- Registration Act 1908 (for lease/rent-agreement registration requirements)
- State-specific eviction procedure rules and notice-period requirements

Domain-specific metadata: applicability_tags

Added to the shared schema's `applicability_tags` field, used for sub-filtering after domain filtering and before semantic search.

- state (mandatory — rent law is almost entirely state-specific in India)
- property_type: residential, commercial
- tenancy_basis: written_agreement, oral, none

Chunking rules specific to this domain

- State is a hard filter dimension here more than any other domain — never let semantic search surface another state's eviction procedure without an explicit state match or an explicit 'general principle' flag.
- Model Tenancy Act 2021 chunks must carry adopted_by_states: list[str] metadata — it is NOT binding law unless a state has enacted it.
- Notice-period and eviction-ground clauses chunked individually per ground (e.g. non-payment, own-use, structural repair) since users usually match on exactly one ground.

Python: domain-specific ingestion notes

```
class TenantChunkMetadata(BaseChunkMetadata):
    state: str # required, not optional, for this domain
    adopted_model_tenancy_act: bool = False

def route_tenant_query(state: str | None):
    if state is None:
        # Force clarification node: cannot safely retrieve without state
        return {"needs_clarification": True, "missing_fact": "state"}
    return {"filter": {"domain": "tenant_property", "state": state}}
```

5. Cyber Crime

Sources to ingest

- Information Technology Act 2000 (as amended 2008)
- IT (Intermediary Guidelines and Digital Media Ethics Code) Rules 2021
- Bharatiya Nyaya Sanhita 2023 provisions relevant to cyber offences (replacing relevant IPC sections)
- Digital Personal Data Protection Act 2023 (and rules once notified — track in_force closely)
- CERT-In advisories and directions (for reporting timelines / procedures)
- Cyber Crime cell / I4C (cybercrime.gov.in) procedural guidance for filing complaints

Domain-specific metadata: applicability_tags

Added to the shared schema's applicability_tags field, used for sub-filtering after domain filtering and before semantic search.

- offence_type: financial_fraud, identity_theft, harassment, data_breach, hacking, defamation
- victim_type: individual, business
- reporting_channel: national_cybercrime_portal, local_police, sectoral_regulator

Chunking rules specific to this domain

- BNS replacing IPC is recent — every criminal-law chunk must carry both old_ipc_section (for legacy judgment matching) and current_bns_section, with in_force clearly set on BNS text.
- DPDP Act 2023: many provisions await notification/rules — flag heavily, do not present as fully operative without checking latest notification status at ingestion time.

- Procedural chunks (how/where to file a cyber complaint) are chunked separately from substantive-law chunks since they're retrieved together but serve different purposes in the response.

Python: domain-specific ingestion notes

```
class CyberChunkMetadata(BaseChunkMetadata):
    old_ipc_section: str | None = None
    current_bns_section: str | None = None
    offence_type: str

def cross_link_ipc_bns(bns_section: dict, ipc_mapping_table: dict) -> dict:
    """Attach legacy IPC section for judgments decided under old law."""
    bns_section["old_ipc_section"] = ipc_mapping_table.get(bns_section["number"])
    return bns_section
```

6. Family & Women's Rights

Sources to ingest

- Protection of Women from Domestic Violence Act 2005
- Hindu Marriage Act 1955 / Special Marriage Act 1954 / Muslim Personal Law (Shariat) Application Act 1937 / Indian Christian Marriage Act 1872 — personal-law fragmentation must be modeled explicitly
- Section 125 CrPC / equivalent BNSS maintenance provisions
- Dowry Prohibition Act 1961
- Sexual Harassment of Women at Workplace (POSH) Act 2013
- Hindu Succession Act 1956 (for inheritance-adjacent family queries)
- Landmark SC/HC judgments on maintenance quantum, custody, POSH internal committee obligations

Domain-specific metadata: applicability_tags

Added to the shared schema's `applicability_tags` field, used for sub-filtering after domain filtering and before semantic search.

- `personal_law_regime`: hindu, muslim, christian, special_marriage_act, other
- `relief_sought`: maintenance, custody, protection_order, workplace_harassment, divorce
- `forum`: family_court, magistrate, internal_committee(POSH)

Chunking rules specific to this domain

- `personal_law_regime` is a mandatory metadata field for marriage/divorce/succession chunks — retrieval must never mix regimes (e.g. serving Hindu Marriage Act provisions for a Muslim personal law query) even if semantically similar wording appears.
- POSH Act chunked separately from general workplace harassment content — POSH has its own procedural machinery (Internal Committee / Local Committee) distinct from criminal-law harassment.
- This domain is the highest sensitivity domain in the system — chunks touching domestic violence must be paired with a ``safety_note`` metadata flag so the response layer can attach helpline info.

Python: domain-specific ingestion notes

```
class FamilyChunkMetadata(BaseChunkMetadata):
    personal_law_regime: str | None = None # required for marriage/divorce/succession doc_types
    safety_sensitive: bool = False         # true for DV Act chunks -> triggers helpline injection downstream

def validate_family_chunk(chunk: dict) -> None:
```

```
if chunk["doc_type"] == "statute" and "marriage" in chunk["act_name"].lower():
    if not chunk.get("personal_law_regime"):
        raise IngestionValidationError(
            f"Missing personal_law_regime for {chunk['act_name']} {chunk['section_number']}"
        )
```

7. End-to-End Python Implementation

Below is a concrete, runnable-shape implementation covering acquisition → parsing → validation → chunking → embedding → storage, generic across all five domains via a pluggable domain config.

7.1 Directory layout

```
ingestion/
  __init__.py
  config.py          # DOMAIN_CONFIGS registry
  models.py          # Pydantic schemas (BaseChunkMetadata + domain subclasses)
  parsers/
    act_parser.py      # bare-act HTML/PDF -> structured JSON tree
    judgment_parser.py
  chunkers/
    statute_chunker.py  # legal-unit chunking, shared logic
    judgment_chunker.py
  validators/
    provenance_validator.py
    domain_validators.py # per-domain rules (e.g. validate_family_chunk)
  embed.py            # OpenAI embedding calls, batching, retry
  store.py            # PGVector upsert
  pipeline.py         # orchestrates the 5 stages end-to-end
  run_ingestion.py    # CLI entrypoint
```

7.2 Domain config registry

```
# ingestion/config.py
from dataclasses import dataclass
from typing import Callable

@dataclass
class DomainConfig:
    domain_id: str
    metadata_model: type
    validators: list[Callable[[dict], None]]
    chunk_size_target: tuple[int, int] = (150, 400) # tokens

DOMAIN_CONFIGS: dict[str, DomainConfig] = {
    "labour_employment": DomainConfig(
        domain_id="labour_employment",
        metadata_model=LabourChunkMetadata,
        validators=[validate_in_force_status],
    ),
    "consumer_protection": DomainConfig(
        domain_id="consumer_protection",
        metadata_model=ConsumerChunkMetadata,
        validators=[validate_in_force_status],
    ),
    "tenant_property": DomainConfig(
        domain_id="tenant_property",
        metadata_model=TenantChunkMetadata,
        validators=[validate_state_present],
    ),
    "cyber_crime": DomainConfig(
        domain_id="cyber_crime",
        metadata_model=CyberChunkMetadata,
        validators=[validate_in_force_status],
    ),
    "family_womens_rights": DomainConfig(
        domain_id="family_womens_rights",
        metadata_model=FamilyChunkMetadata,
        validators=[validate_family_chunk],
    ),
}
```

7.3 Structural parsing (Act -> JSON tree)

```
# ingestion/parsers/act_parser.py
import re
from bs4 import BeautifulSoup

SECTION_RE = re.compile(r"^(\d+[A-Z]?)\.\s+(.*)$")

def parse_bare_act(html: str, act_name: str, act_year: int) -> dict:
    soup = BeautifulSoup(html, "html.parser")
    sections = []
    current_chapter = None

    for el in soup.select(".chapter, .section"):
        if "chapter" in el.get("class", []):
            current_chapter = el.get_text(strip=True)
            continue

        raw = el.get_text(" ", strip=True)
        match = SECTION_RE.match(raw)
        if not match:
            continue

        number, rest = match.groups()
        title, _, body = rest.partition(" - ") if " - " in rest else (rest, "", "")
        sections.append({
            "number": number,
            "title": title.strip(),
            "chapter": current_chapter,
            "text": body.strip() or rest,
            "sub_sections": extract_sub_sections(el),
            "provisos": extract_provisos(el),
            "cross_references": extract_cross_references(raw),
        })

    return {"act_name": act_name, "act_year": act_year, "sections": sections}

def extract_sub_sections(el) -> list[dict]:
    out = []
    for i, sub in enumerate(el.select(".sub-section")):
        out.append({
            "label": f"({chr(97 + i)})",
            "text": sub.get_text(" ", strip=True),
            "provisos": [p.get_text(" ", strip=True) for p in sub.select(".proviso")],
        })
    return out

def extract_provisos(el) -> list[str]:
    return [p.get_text(" ", strip=True) for p in el.select(".proviso")]

def extract_cross_references(text: str) -> list[str]:
    return re.findall(r"[Ss]ection\s+\d+[A-Z]?(?:\s+(\w+))?", text)
```

7.4 Provenance + validation gate

```
# ingestion/validators/provenance_validator.py
from datetime import date

REQUIRED_FIELDS = [
    "act_name", "jurisdiction", "in_force", "source_url", "last_verified"
]

class IngestionValidationError(Exception):
    pass
```

```

def validate_provenance(chunk: dict) -> None:
    missing = [f for f in REQUIRED_FIELDS if chunk.get(f) in (None, "")]
    if missing:
        raise IngestionValidationError(
            f"Chunk {chunk.get('chunk_id')} missing required provenance fields: {missing}"
        )
    if chunk["jurisdiction"] == "state" and not chunk.get("state"):
        raise IngestionValidationError(
            f"Chunk {chunk.get('chunk_id')} is state jurisdiction but has no state set"
        )
    if (date.today() - chunk["last_verified"]).days > 365:
        raise IngestionValidationError(
            f"Chunk {chunk.get('chunk_id')} source not re-verified in over a year - re-check before ingest"
        )

```

7.5 Legal-unit chunker (shared core, used by every domain)

```

# ingestion/chunkers/statute_chunker.py
import uuid
from ingestion.models import BaseChunkMetadata

def token_count(text: str) -> int:
    # swap for tiktoken in production
    return len(text.split())

def make_chunk(header: str, body: str, section: dict, act_meta: dict,
               metadata_model=BaseChunkMetadata, **extra_fields) -> dict:
    chunk_text = f"{header}\n\n{body}".strip()
    metadata = metadata_model(
        chunk_id=str(uuid.uuid4()),
        domain=act_meta["domain"],
        doc_type="statute",
        act_name=act_meta["act_name"],
        act_year=act_meta.get("act_year"),
        section_number=section["number"],
        section_title=section["title"],
        jurisdiction=act_meta["jurisdiction"],
        state=act_meta.get("state"),
        in_force=act_meta.get("in_force", True),
        superseded_by=act_meta.get("superseded_by"),
        source_url=act_meta["source_url"],
        last_verified=act_meta["last_verified"],
        applicability_tags=act_meta.get("applicability_tags", []),
        **extra_fields,
    )
    return {"text": chunk_text, "metadata": metadata.model_dump()}

def chunk_statute_section(section: dict, act_meta: dict, config) -> list[dict]:
    header = f"{act_meta['act_name']} ({act_meta.get('act_year', '')}), Section {section['number']} - {section['title']}"
    chunks = []

    if token_count(section["text"]) < config.chunk_size_target[1]:
        body = section["text"] + "\n" + "\n".join(section.get("provisos", []))
        chunks.append(make_chunk(header, body, section, act_meta, config.metadata_model))
    else:
        for sub in section["sub_sections"]:
            body = sub["text"] + "\n" + "\n".join(sub.get("provisos", []))
            chunks.append(make_chunk(f"{header} {sub['label']}", body, section, act_meta, config.metadata_model))

    # Definitions section: one chunk per defined term
    if section["number"] == "2" and section.get("sub_sections"):
        chunks = [
            make_chunk(f"{act_meta['act_name']} - Definition {s['label']}", s["text"], section, act_meta, config.metadata_model)
            for s in section["sub_sections"]
        ]
    return chunks

```

7.6 Embedding

```
# ingestion/embed.py
import time
from openai import OpenAI

client = OpenAI()
EMBED_MODEL = "text-embedding-3-small"
BATCH_SIZE = 96

def embed_chunks(chunks: list[dict]) -> list[dict]:
    for i in range(0, len(chunks), BATCH_SIZE):
        batch = chunks[i:i + BATCH_SIZE]
        texts = [c["text"] for c in batch]

        for attempt in range(3):
            try:
                resp = client.embeddings.create(model=EMBED_MODEL, input=texts)
                break
            except Exception:
                if attempt == 2:
                    raise
                time.sleep(2 ** attempt)

        for chunk, item in zip(batch, resp.data):
            chunk["embedding"] = item.embedding

    return chunks
```

7.7 PGVector storage

```
# ingestion/store.py
import json
import psycopg2
from psycopg2.extras import execute_values

DDL = """
CREATE TABLE IF NOT EXISTS legal_chunks (
    chunk_id UUID PRIMARY KEY,
    domain TEXT NOT NULL,
    doc_type TEXT NOT NULL,
    act_name TEXT,
    section_number TEXT,
    jurisdiction TEXT NOT NULL,
    state TEXT,
    in_force BOOLEAN NOT NULL,
    superseded_by TEXT,
    source_url TEXT NOT NULL,
    last_verified DATE NOT NULL,
    applicability_tags TEXT[],
    text TEXT NOT NULL,
    metadata JSONB NOT NULL,
    embedding VECTOR(1536) NOT NULL
);
CREATE INDEX IF NOT EXISTS idx_chunks_domain ON legal_chunks (domain);
CREATE INDEX IF NOT EXISTS idx_chunks_state ON legal_chunks (state);
CREATE INDEX IF NOT EXISTS idx_chunks_in_force ON legal_chunks (in_force);
CREATE INDEX IF NOT EXISTS idx_chunks_embedding ON legal_chunks
    USING ivfflat (embedding vector_cosine_ops) WITH (lists = 100);
"""

def upsert_chunks(conn, chunks: list[dict]) -> None:
    with conn.cursor() as cur:
        cur.execute(DDL)
        rows = [
            (
                c["metadata"]["chunk_id"], c["metadata"]["domain"], c["metadata"]["doc_type"],
```

```

        c["metadata"].get("act_name"), c["metadata"].get("section_number"),
        c["metadata"]["jurisdiction"], c["metadata"].get("state"),
        c["metadata"]["in_force"], c["metadata"].get("superseded_by"),
        c["metadata"]["source_url"], c["metadata"]["last_verified"],
        c["metadata"].get("applicability_tags", []),
        c["text"], json.dumps(c["metadata"]), c["embedding"],
    )
    for c in chunks
]
execute_values(cur, """
INSERT INTO legal_chunks (
    chunk_id, domain, doc_type, act_name, section_number, jurisdiction,
    state, in_force, superseded_by, source_url, last_verified,
    applicability_tags, text, metadata, embedding
) VALUES %s
ON CONFLICT (chunk_id) DO UPDATE SET
    text = EXCLUDED.text, metadata = EXCLUDED.metadata,
    embedding = EXCLUDED.embedding, last_verified = EXCLUDED.last_verified
""", rows)
conn.commit()

```

7.8 Orchestration pipeline

```

# ingestion/pipeline.py
from ingestion.config import DOMAIN_CONFIGS
from ingestion.parsers.act_parser import parse_bare_act
from ingestion.chunkers.statute_chunker import chunk_statute_section
from ingestion.validators.provenance_validator import validate_provenance, IngestionValidationError
from ingestion.embed import embed_chunks
from ingestion.store import upsert_chunks

def ingest_act(html: str, act_meta: dict, conn) -> dict:
    config = DOMAIN_CONFIGS[act_meta["domain"]]
    parsed = parse_bare_act(html, act_meta["act_name"], act_meta.get("act_year"))

    all_chunks, failures = [], []
    for section in parsed["sections"]:
        for chunk in chunk_statute_section(section, act_meta, config):
            try:
                validate_provenance(chunk["metadata"])
                for validator in config.validators:
                    validator(chunk["metadata"])
                all_chunks.append(chunk)
            except IngestionValidationError as e:
                failures.append({"chunk": chunk, "error": str(e)})

    if failures:
        # Fail loudly rather than silently embedding unverified law
        raise IngestionValidationError(
            f"{len(failures)} chunk(s) failed validation for {act_meta['act_name']}: "
            f"{[f['error'] for f in failures]}"
        )

    embedded = embed_chunks(all_chunks)
    upsert_chunks(conn, embedded)
    return {"act": act_meta["act_name"], "chunks_ingested": len(embedded)}

```

7.9 CLI entrypoint

```

# ingestion/run_ingestion.py
import argparse, psycpg2, requests
from ingestion.pipeline import ingest_act

def main():
    parser = argparse.ArgumentParser()
    parser.add_argument("--source-url", required=True)
    parser.add_argument("--domain", required=True, choices=[

```

```

        "labour_employment", "consumer_protection", "tenant_property",
        "cyber_crime", "family_womens_rights"
    ])
    parser.add_argument("--act-name", required=True)
    parser.add_argument("--jurisdiction", required=True, choices=["central", "state"])
    parser.add_argument("--state", default=None)
    args = parser.parse_args()

    html = requests.get(args.source_url, timeout=30).text
    conn = psycopg2.connect(dsn="postgresql://...")

    result = ingest_act(html, {
        "domain": args.domain,
        "act_name": args.act_name,
        "jurisdiction": args.jurisdiction,
        "state": args.state,
        "source_url": args.source_url,
        "in_force": True,
        "last_verified": __import__("datetime").date.today(),
    }, conn)

    print(result)

if __name__ == "__main__":
    main()

```

8. Retrieval-Time Filtering (how chunking pays off)

The chunking and metadata design above exists to make this filter step cheap and correct — domain and sub-domain metadata filtering always runs BEFORE semantic (vector) search.

```
def build_retrieval_filter(domain: str, facts: dict) -> dict:
    f = {"domain": domain, "in_force": True}

    if domain == "tenant_property":
        if not facts.get("state"):
            raise ValueError("state is required for tenant_property retrieval")
        f["state"] = facts["state"]

    if domain == "family_womens_rights" and facts.get("personal_law_regime"):
        f["personal_law_regime"] = facts["personal_law_regime"]

    if domain == "labour_employment" and facts.get("state"):
        # labour retrieval includes BOTH central acts AND the matching state's rules
        f["jurisdiction__in"] = ["central", "state"]
        f["state__in"] = [None, facts["state"]]

    return f

def hybrid_retrieve(query_embedding, filters: dict, top_k: int = 8):
    sql_filter, params = compile_filters_to_sql(filters)
    return db.execute(f"""
        SELECT chunk_id, text, metadata, 1 - (embedding <=> %s) AS score
        FROM legal_chunks
        WHERE {sql_filter}
        ORDER BY embedding <=> %s
        LIMIT %s
    """, params + [query_embedding, query_embedding, top_k])
```

9. Ingestion Checklist Summary

Domain	Hard filter dimension	Highest ingestion risk
Labour & Employment	in_force (old Acts vs new Codes)	Citing Labour Code provisions not yet notified
Consumer Protection	complaint value bracket / forum	Wrong forum jurisdiction (District/State/National)
Tenant & Property	state (mandatory)	Cross-state law leakage via semantic similarity
Cyber Crime	in_force (IPC → BNS transition)	Mixing repealed IPC sections with current BNS
Family & Women's Rights	personal_law_regime	Cross-regime leakage (e.g. Hindu law served for Muslim query)

This document is an implementation plan for internal engineering use. It does not itself constitute legal advice, and all ingested sources must be verified against the official India Code / e-Gazette at time of ingestion, not relied upon from this document.